vAPI API Penetration Testing Report

Tester: Charmaine Mangorima
Date: 20 February 2026
Target Application: vAPI (Hosted locally running on Docker)
API Version: 1.1
Server: http://localhost:8000/vapi
Test Type: API Penetration Test
Environment: Local Docker Compose Deployment

Executive Summary

The objective of this security assessment was to perform a focused penetration test of the **vAPI application**, specifically targeting **API endpoints 6 to 10**, as defined in the provided Swagger/OpenAPI specification. The goal was to identify, analyse and exploit such as mass assignment, security misconfiguration, injection, improper assets management, lack of logging and monitoring and to document each weakness with evidence and practical exploitation steps.

Scope

In-scope

- vAPI application hosted at: **http://localhost:8000/vapi**
- API endpoints **6 to 10** (as provided in Swagger/OpenAPI file)
- Testing of:
    - Mass assignment
    - Security misconfiguration
    - Injection
    - Improper assets management
    - Lack of logging and monitoring

Out-of-scope

- Testing beyond endpoints 6-10
- Attacks against the hosting provider infrastructure
- Any persistence, malware or destructive exploitation beyond required flags

Tools used

- Postman
- Docker Desktop
- Burpsuite
- sqlmap
- Cursor

Methodology

Testing was performed against the locally hosted vAPI instance running in Docker Compose, accessible at http://localhost:8000/vapi. The assessment followed a structured API penetration testing approach aligned with the **OWASP API Security Top 10**, focusing specifically on endpoints **API6 - API10** as defined in the provided Swagger/OpenAPI (OAS 3.0) specification.

The methodology consisted of the following phases:

1. Reconnaissance & Documentation Review

- Reviewed the Swagger/OpenAPI specification

2. Environment Setup

- Confirmed the API was running correctly in Docker and accessible via the base URL.

- Set up Postman to Burpsuite proxy

3. Evidence Collection & Validation

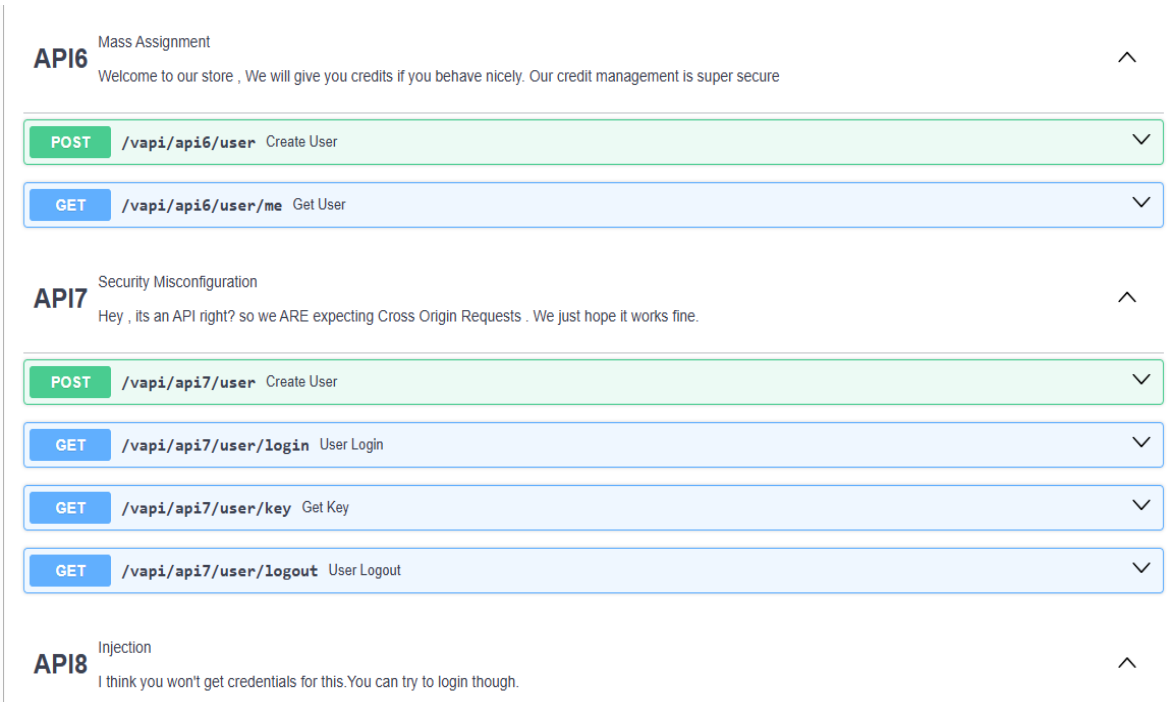
- Each finding was validated by:

    - Capturing request/response traffic in Burp Suite and Postman

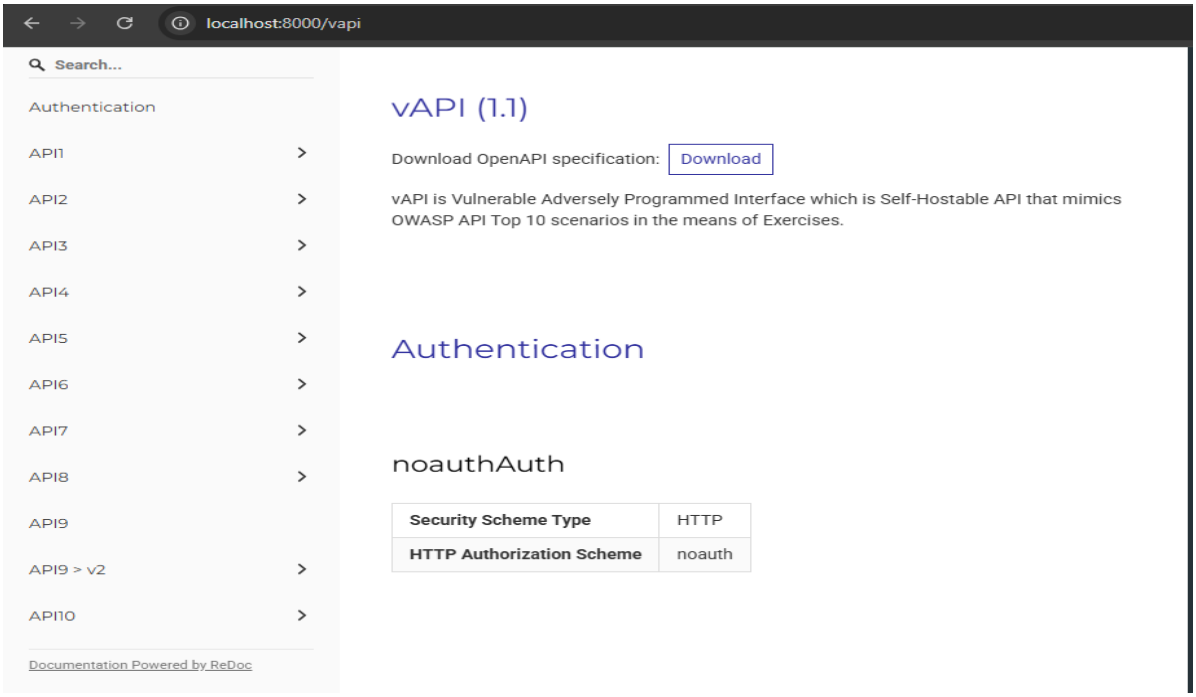
    - Extracting and recording flags as proof of exploitation

- Results were documented with clear reproduction steps and mapped to OWASP API Security categories.

Swagger/OpenAPI (OAS 3.0) specification



The application running on localhost:8000/vapi



Findings

Finding 1: Mass assignment

OWASP Mapping: API3: Broken Object Property Level Authorization

Endpoint: POST /vapi/api6/user

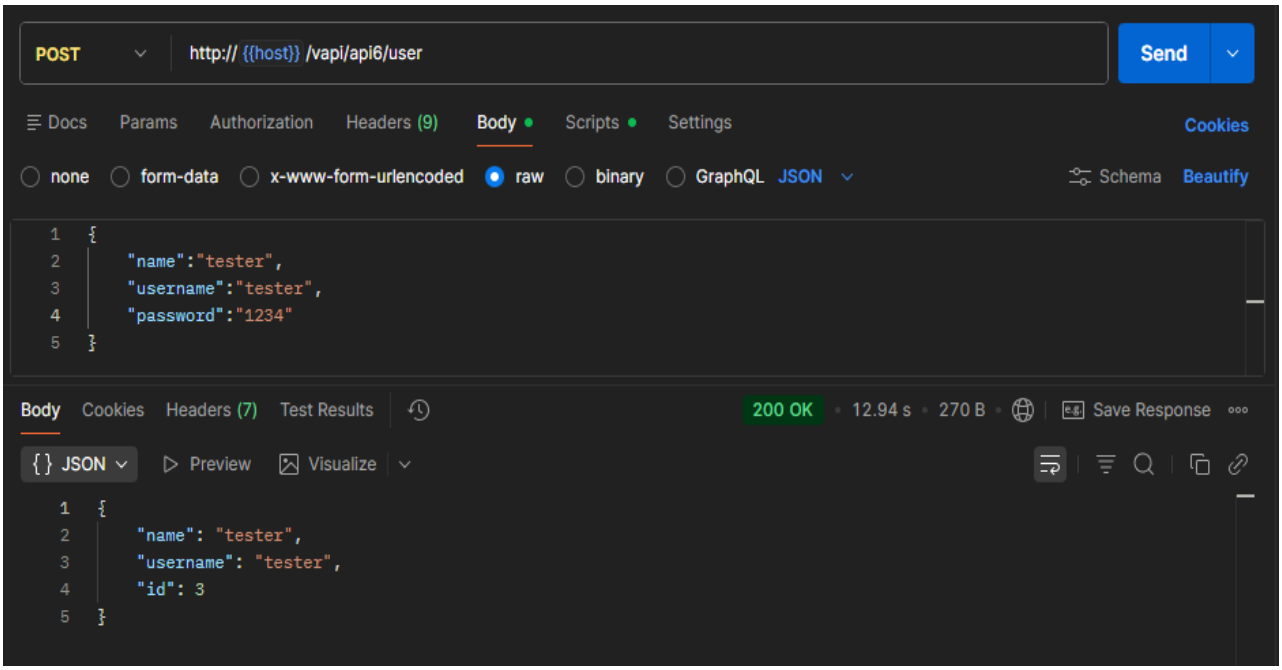
Description

APIs trust user input received from the client-side and directly bind it to backend objects without filtering or restricting allowed parameters. An attacker can identify hidden or sensitive properties through API documentation, educated guesses or analyzing HTTP responses and inject additional parameters into the request to gain unauthorized access to restricted functionalities.

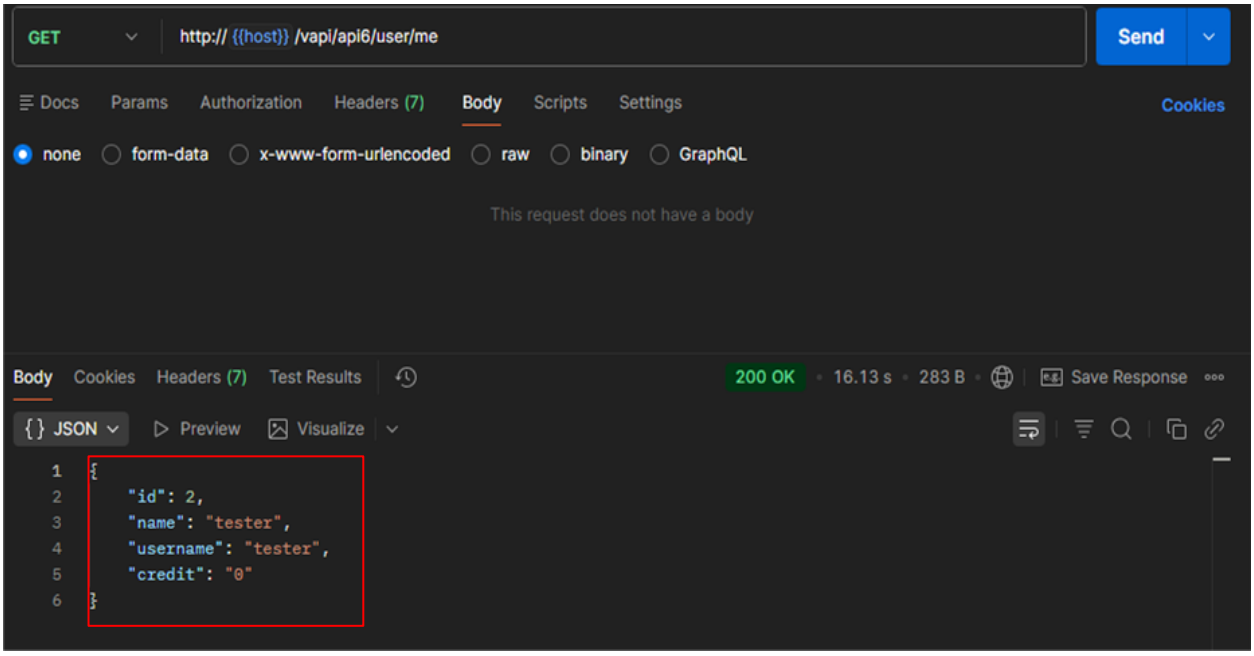
For example, while registering a user, an attacker can append parameters such as "isAdmin": true in the JSON body and potentially gain elevated privileges or manipulate account balances.

Evidence

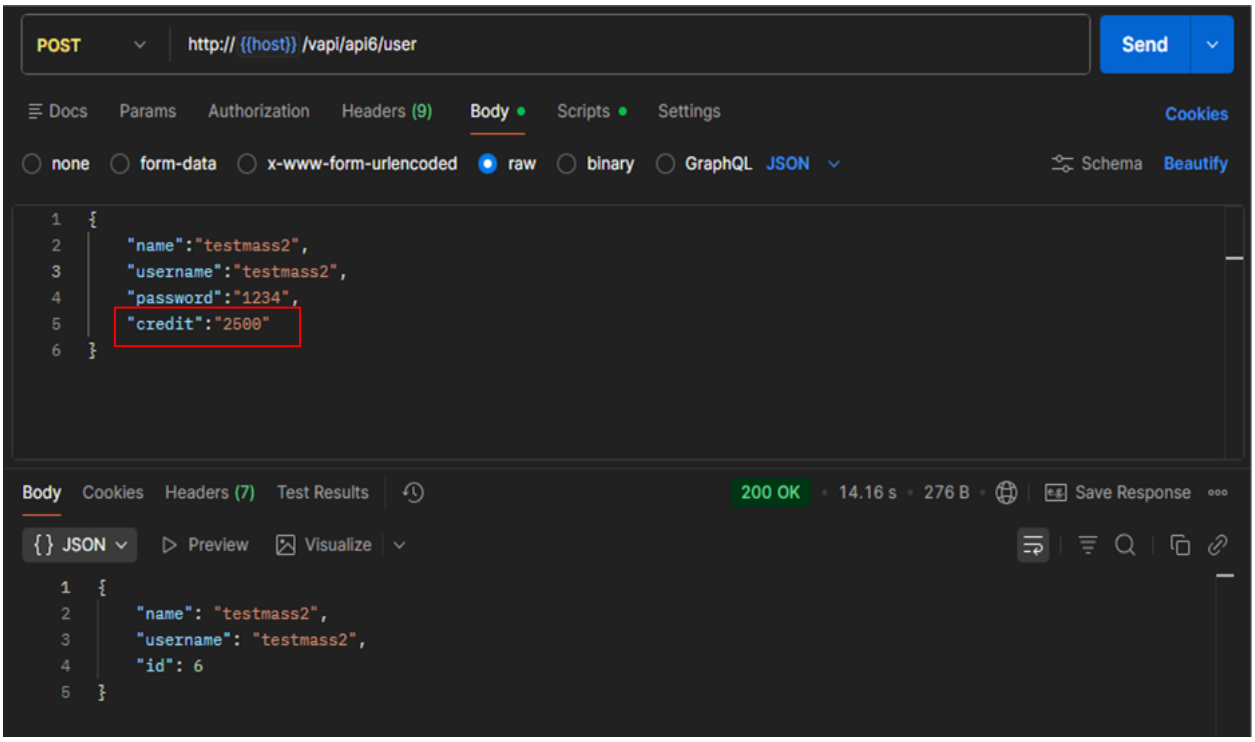
I created a user using *POST /vapi/api6/user* endpoint by providing username and password.



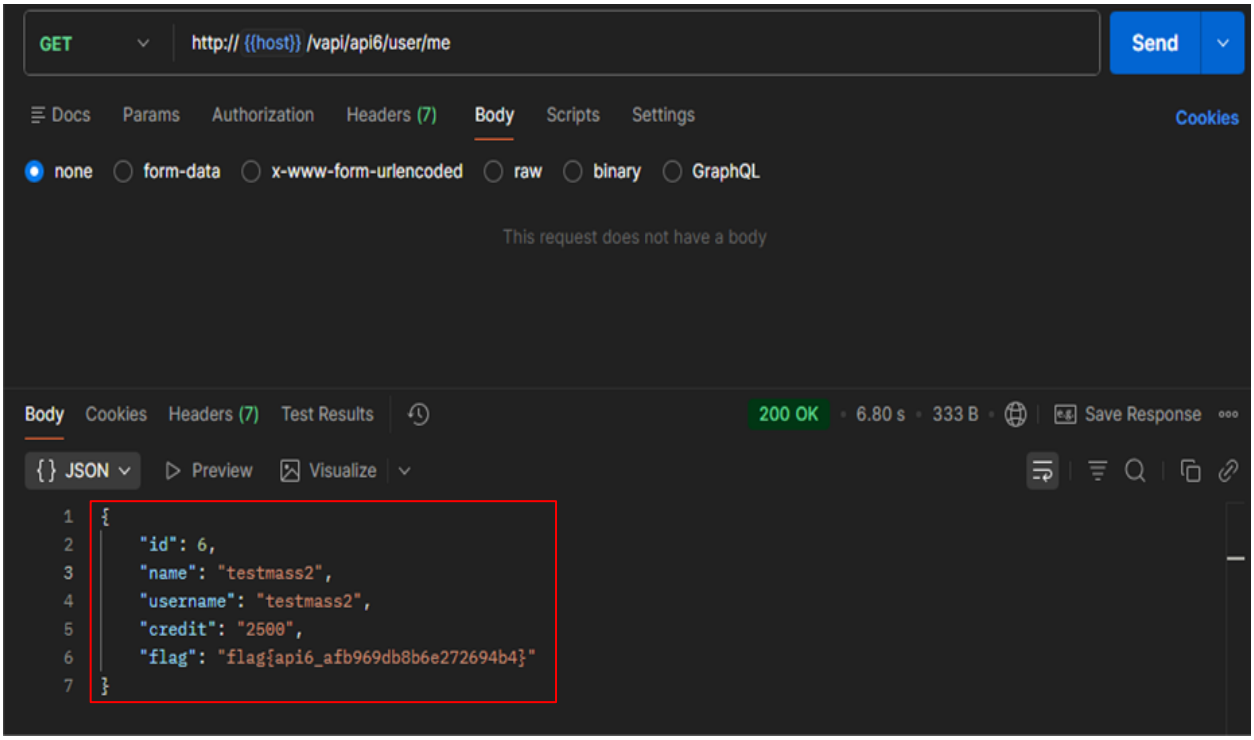
I retrieved the new user using the *GET /vapi/api6/user/me.* The response shows extra fields like the credit which is expected to be a value that the client should not be able to manipulate.



I tried to create a new user and check if the Post endpoint is vulnerable to mass assignment by adding the credit value to the request body.



I retrieved the new user details using the GET endpoint. The response returns the new user details including the credit. This shows that the API does not validate user input to check if there are extra fields added by the user.



Flag found: flag{api6_afb969db8b6e272694b4}

Impact

- Privilege escalation
- Unauthorized manipulation of business logic
- Financial abuse (credit tampering)
- Full compromise of user access control

Remediation

- Implement server-side allowlists for permitted fields.
- Use Data Transfer Objects instead of directly binding request bodies to models.
- Explicitly ignore sensitive properties from user input.
- Apply schema validation and input validation on the backend.

Finding 2: Security misconfiguration

OWASP Mapping: API8: Security Misconfiguration

Endpoint: GET /vapi/api7/user/key

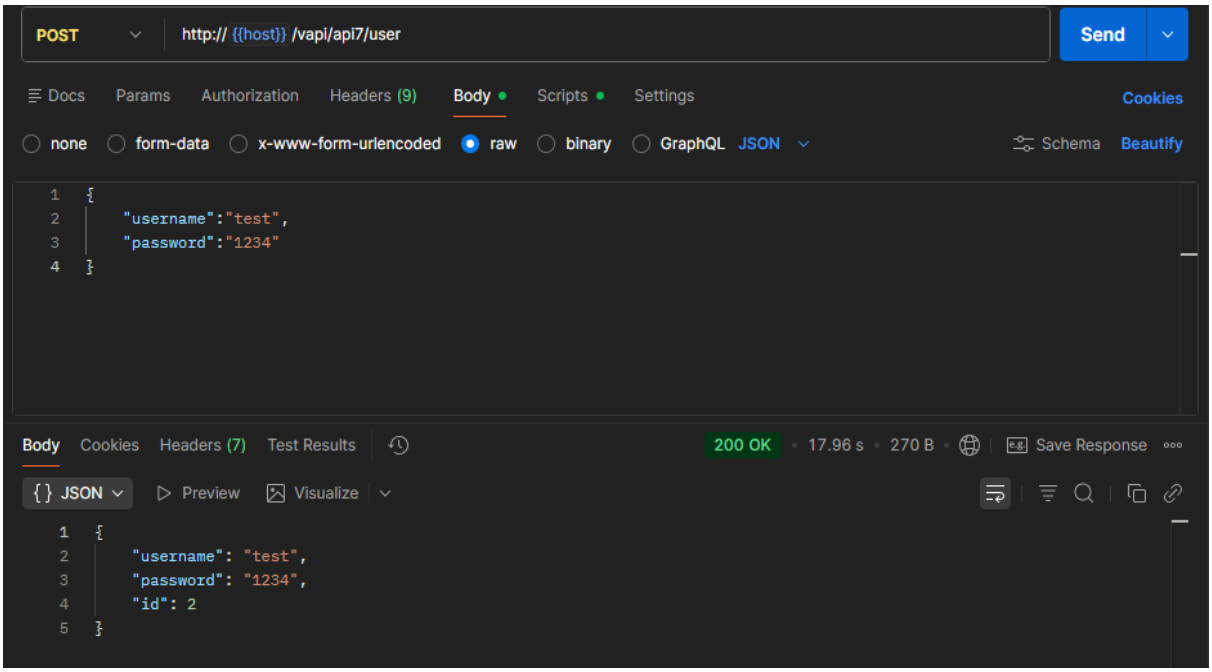
Description

Security misconfiguration occurs when APIs are deployed with insecure default settings, verbose error messages containing sensitive data, unnecessary endpoints enabled, or improper access control mechanisms. Sensitive endpoints may be exposed without authentication and misconfigured CORS policies may allow requests from untrusted origins.

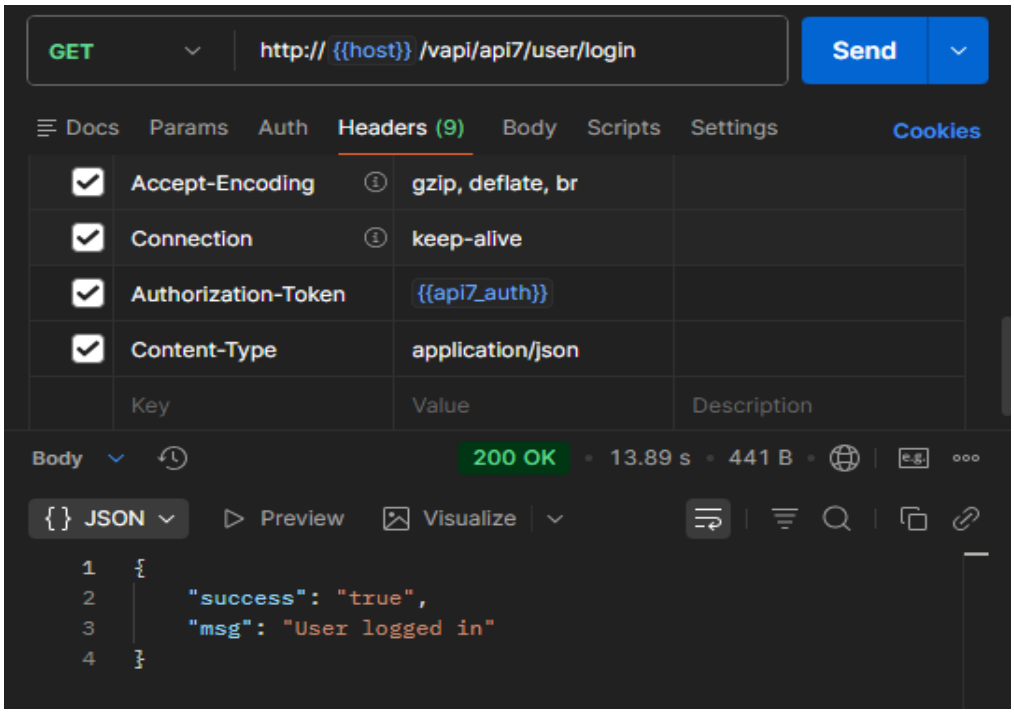
An attacker may exploit these poor configurations to gain unauthorized access or break security mechanisms and it allows any third-party web application to send authenticated requests and receive sensitive information.

Evidence

I created a new user using the *POST /vapi/api7/user* endpoint by providing username and password



I successfully logged in as the new user

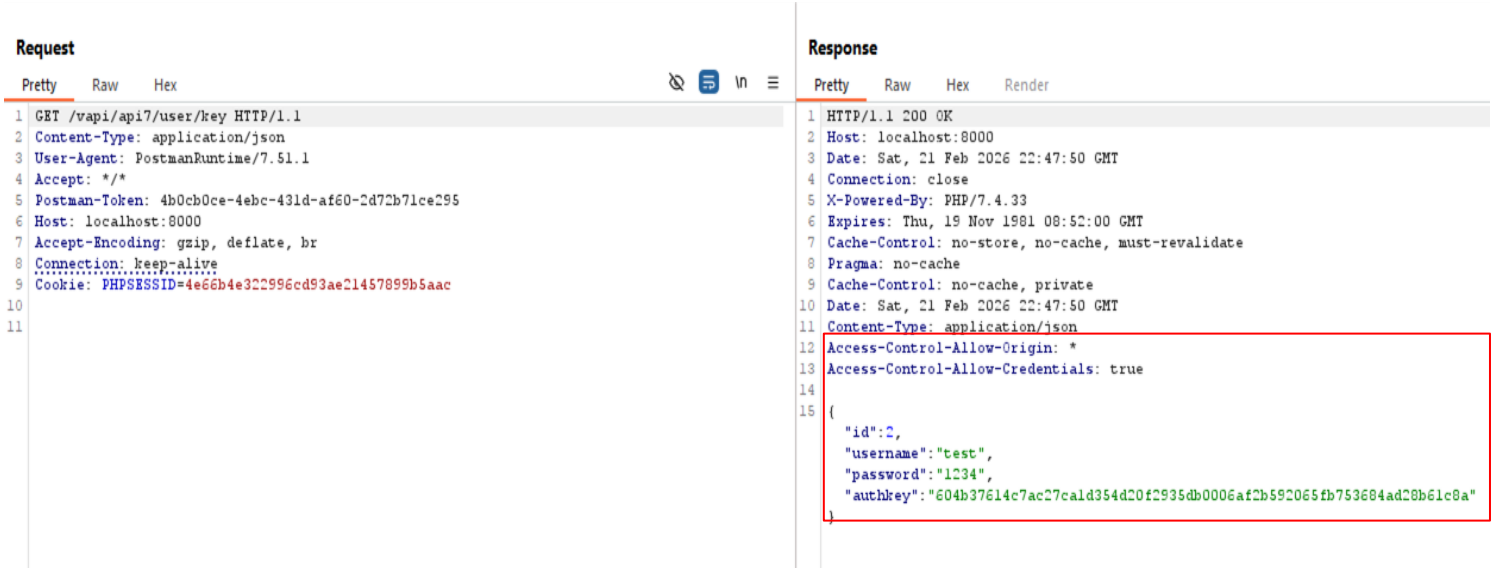


After logging in I sent a request to the *GET /vapi/api7/user/key* to retrieve the auth key and user details.

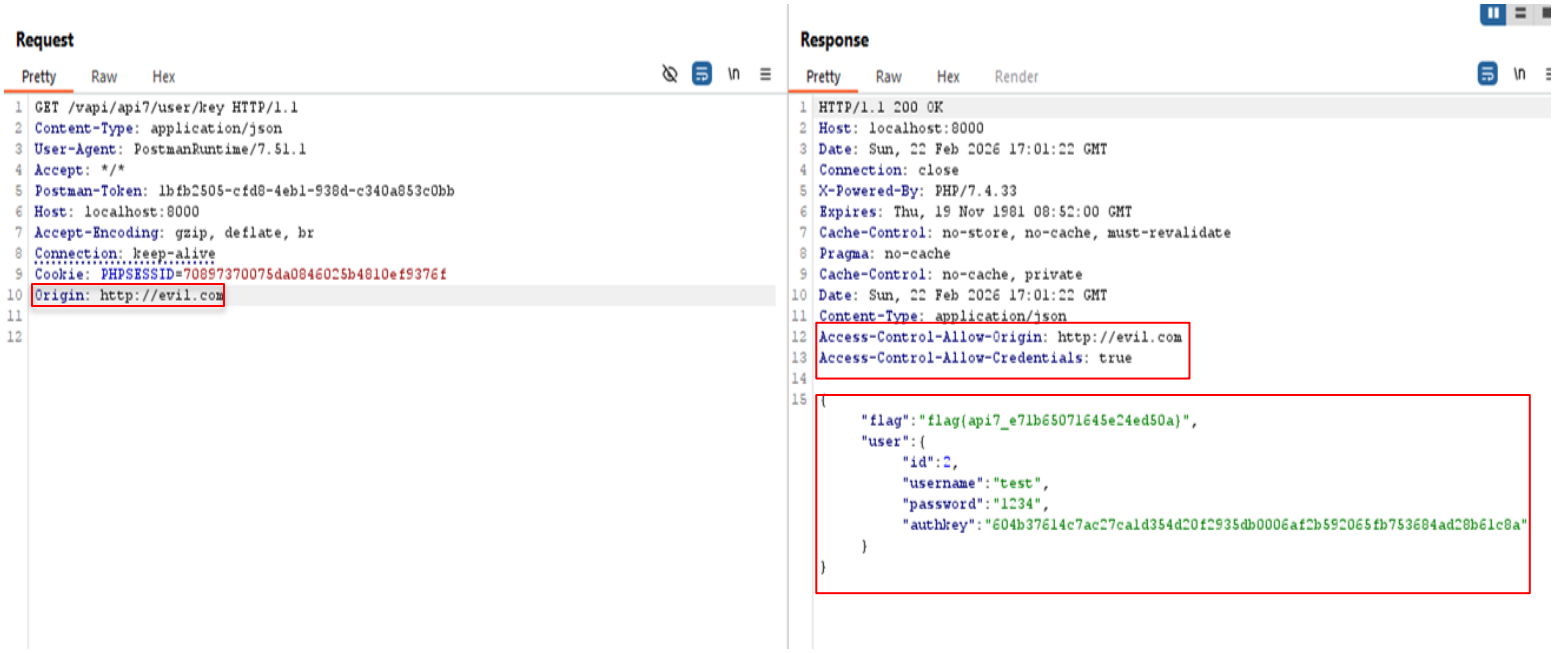
I intercepted the request in Burpsuite and sent to Repeater

The response headers contained a permissive CORS configurations : **Access-Control-Allow-Origin: *** and **Access-Control-Allow-Credentials: true**

This confirms improper access control and potential CORS misconfiguration.



I added a custom Origin header with value *http://evil.com* to the request and sent it again.The API successfully returned a response showing **Access-Control-Allow-Origin: http://evil.com**



This way an attacker can have access to sensitive user data like the username, password and auth key and the user account will be compromised.

Flag found: flag{api7_e71b65071645e24ed50a}

Impact

- Stealing API keys
- Stealing session tokens
- Accessing user data
- Performing actions on behalf of victim

Remediation

- Never use Access-Control-Allow-Origin: * with credentials
- Explicitly whitelist trusted domains

- Disable credentials if not needed
- Validate authentication server-side and never rely on CORS alone
- Perform secure configuration reviews before deployment.

Finding 3: Injection

OWASP Mapping: API8: Security Misconfiguration

Endpoint: POST /vapi/api8/user/login

Description

Injection vulnerabilities occur when APIs fail to properly validate or sanitize user input before processing it in backend queries or commands. If user-supplied input is directly concatenated into SQL queries, attackers can manipulate query logic to bypass authentication.

For example, an attacker may supply SQL injection payloads in the login request to bypass authentication controls.

Evidence

Firstly, I sent a request to the *POST /vapi/api8/user/login* and intercepted the request in Burpsuite and sent it to Repeater. I copied to file and saved the request as ***login.txt***

login - Notepad

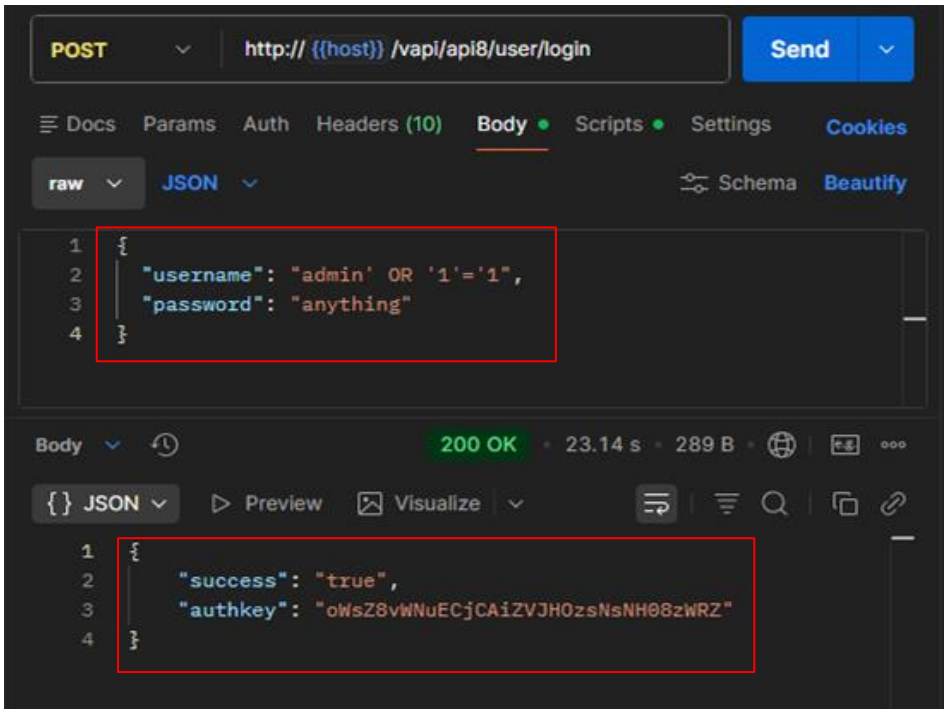
File Edit Format View Help

```
POST /vapi/api8/user/login HTTP/1.1
Content-Type: application/json
User-Agent: PostmanRuntime/7.51.1
Accept: */*
Postman-Token: 3b256d9f-1e14-4c2f-91da-b0bfa28b6acd
Host: localhost:8000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 49
Cookie: PHPSESSID=43120181df8dc06151cb53b8bfc95b37

{
  "username": "test",
  "password": "test"
}
```

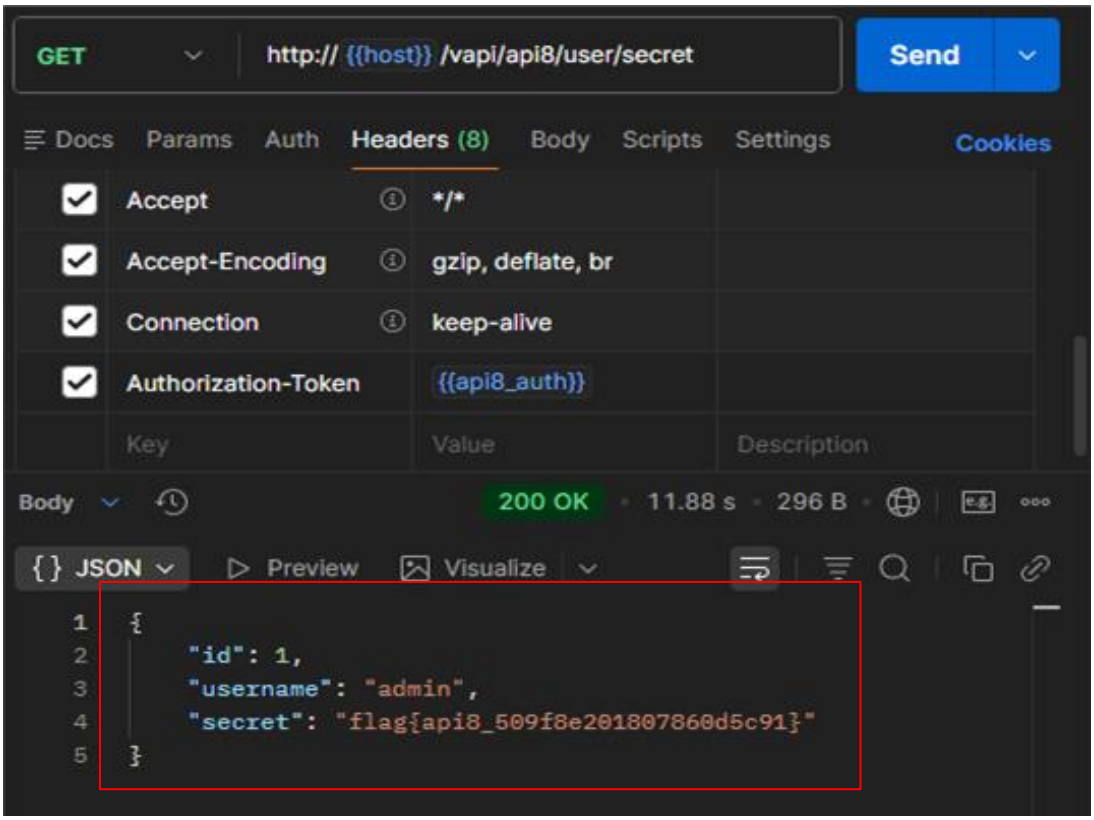
I used sqlmap and confirmed that the endpoint is vulnerable to injection and specifically the username parameter in the request payload. I also confirmed that the database is MYSQL.

```
charmaine@GirlBoss:/mnt/c/Users/HP/Documents$ sqlmap -r login.txt
        ___
       __H__
 ___ ___[']_____ ___ ___  {1.8.4#stable}
|_ -| . ["]     | .'| . |
|___|_  ["]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility t
tate and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:33:21 /2026-02-22/

[19:33:21] [INFO] parsing HTTP request from 'login.txt'
JSON data found in POST body. Do you want to process it? [Y/n/q] y
[19:33:24] [INFO] testing connection to the target URL
[19:33:27] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[19:33:27] [INFO] testing if the target URL content is stable
[19:33:33] [INFO] target URL content is stable
[19:33:33] [INFO] testing if (custom) POST parameter 'JSON username' is dynamic
[19:33:35] [WARNING] (custom) POST parameter 'JSON username' does not appear to be dynamic
[19:33:40] [INFO] heuristic (basic) test shows that (custom) POST parameter 'JSON username' might be injectable (possible DBMS: 'MySQL')
[19:33:44] [INFO] testing for SQL injection on (custom) POST parameter 'JSON username'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[19:33:57] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[19:34:12] [WARNING] reflective value(s) found and filtering out
[19:34:26] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[19:34:33] [INFO] testing 'Generic inline queries'
[19:34:36] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[19:37:22] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[19:40:05] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
```

In postman I tested injection by using a MYSQL injection payload to the *POST /vapi/api8/user/login* endpoint. I successfully logged in as the admin and got an auth key.



I went ahead and sent a request to the *GET /vapi/api8/user/secret* endpoint. The API returned a response that includes the secret that is the flag.



Flag found: flag{api8_509f8e201807860d5c91}

Impact

- Authentication bypass
- Unauthorized access to sensitive data
- Database compromise
- Data exfiltration

Remediation

- Use parameterized queries or prepared statements.
- Avoid dynamic SQL query construction.
- Implement strict input validation and sanitization.
- Suppress detailed error messages.

Finding 4: Improper assets management

OWASP Mapping: API9: Improper Inventory Management

Endpoint: POST */vapi/api9/v2/user/login*

Description

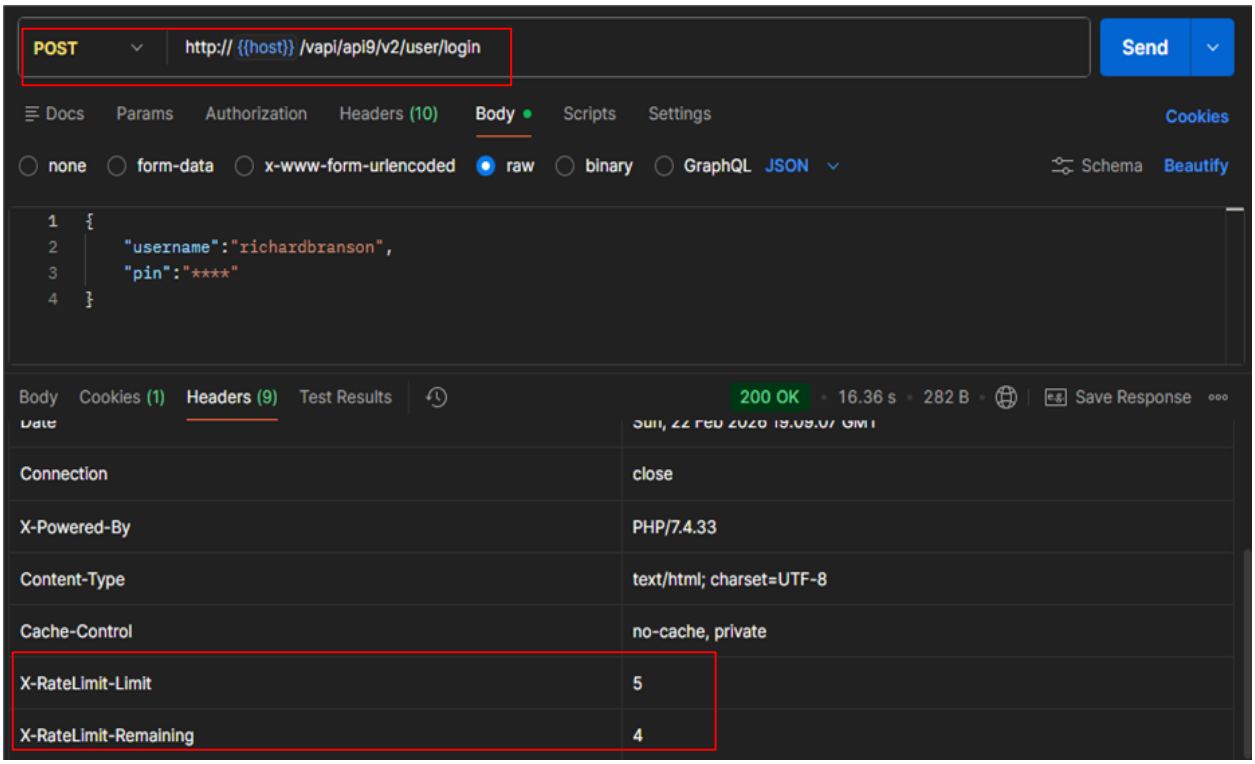
Improper assets management occurs when outdated or undocumented API versions remain accessible, increasing the attack surface. Attackers can enumerate API versions and discover deprecated endpoints that may contain weaker security controls.

For example, the presence of */vapi/api9/v2/user/login* suggests multiple API versions may be accessible without proper lifecycle management.

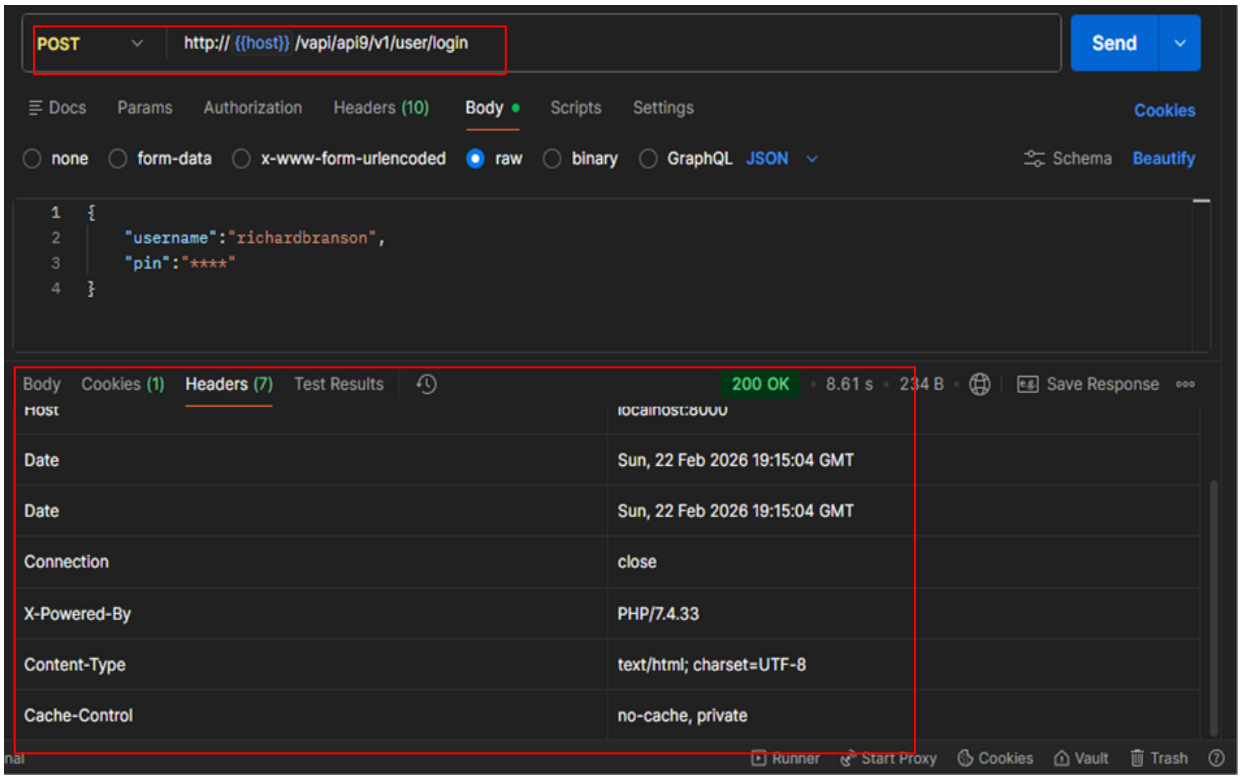
Evidence

I sent a request to the v2 login endpoint with the default payload. In the response headers, I discovered that rate limiting was implemented in v2 and it was set to 5.

This makes it difficult to bruteforce the four digit pin.



I then did the same for the v1 login endpoint. No rate limiting was implemented in v1 and the v1 of the API is still live.



I used the 0000-9999 wordlist from SecLists to bruteforce the 4 digit pin using Burpsuite Intruder.

All responses were successful with status code 200 but the login was successful with the pin **1655.** It was the only request that had a different response length and that is why I decided to check it out.

Payload	Status code	Response received	Error	Timeout	Length
1650	200	4908			234
1651	200	4066			234
1652	200	4977			234
1653	200	5143			234
1654	200	4306			234
1655	200	4988			309
1656	200	3913			234
1657	200	5128			234
1658	200	4501			234
1659	200	3995			234
1660	200	4650			234

Request Response

Pretty Raw Hex

```
1  POST /vapi/api9/v1/user/login HTTP/1.1
2  Content-Type: application/json
3  User-Agent: PostmanRuntime/7.51.1
4  Accept: */*
5  Postman-Token: e2081f7a-175c-4471-8697-46f521dd9ea2
6  Host: localhost:8000
7  Accept-Encoding: gzip, deflate, br
8  Connection: keep-alive
9  Content-Length: 56
10
11 {
12   "username":"richardbranson",
13   "pin":"1655"
14 }
```

Request Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Host: localhost:8000
3  Date: Sun, 22 Feb 2026 20:43:15 GMT
4  Connection: close
5  X-Powered-By: PHP/7.4.33
6  Cache-Control: no-cache, private
7  Date: Sun, 22 Feb 2026 20:43:15 GMT
8  Content-Type: application/json
9
10 {
     "id":1,
     "username":"richardbranson",
     "accbalance":"flag{api9_81e306bdd20a7734e244}"
   }
```

Flag found: flag{api9_81e306bdd20a7734e244}

Impact

- Exposure of legacy vulnerabilities
- Bypass of updated security controls
- Data leakage from outdated APIs
- Increased attack surface

Remediation

- Maintain a complete API inventory.
- Deprecate and remove old API versions.
- Enforce authentication consistently across versions.
- Use API gateways to manage and monitor versions.
- Conduct periodic asset discovery scans.

Finding 5: Lack of logging and monitoring

OWASP Mapping: API8: Security Miscofiguration

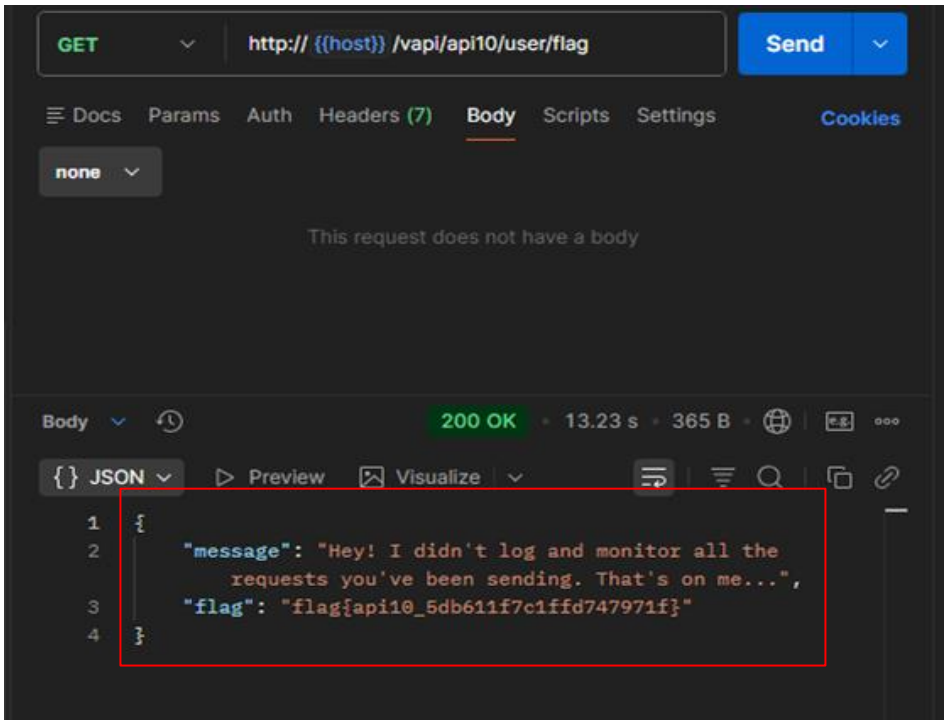
Endpoint: GET /vapi/api10/user/flag

Description

Lack of logging and monitoring means that the developer does not implement sufficient logging and monitoring on the API. This leaves room for missed security alerts and proper response to security incidents on the API.

Evidence

I sent a request to the GET /vapi/api10/user/flag endpoint and the response shows a message that the whole API lacks monitoring and monitoring.



Flag found: flag{api10_5db611f7c1ffd747971f}

Impact

- Delayed breach detection

- Prolonged attacker persistence
- Increased data exfiltration risk
- Regulatory compliance violations

Remediation

- Implement centralized logging (SIEM integration).
- Log authentication attempts and suspicious behavior.
- Configure alerting for abnormal activity.
- Implement rate limiting and anomaly detection.
- Regularly review logs and conduct security monitoring.

Conclusion

The API penetration test of the locally hosted vAPI application revealed multiple security weaknesses across endpoints 6 to 10, corresponding to several categories of the OWASP API Security Top 10. The assessment identified critical vulnerabilities, including mass assignment, security misconfiguration, injection, improper assets management and lack of logging and monitoring. Each vulnerability was verified with practical exploitation and evidence in the form of flags was successfully collected.

These findings indicate that the application currently exposes sensitive functionality and data to unauthorized users, increasing the risk of privilege escalation, data exfiltration, business logic manipulation and unauthorized access. The presence of legacy API versions and permissive CORS configurations further expands the attack surface.

To mitigate these risks, it is recommended that the development team implement strict input validation, enforce access controls, maintain a complete API inventory, remove deprecated endpoints and establish comprehensive logging and monitoring. Addressing these issues will significantly improve the security posture of the vAPI application and reduce the likelihood of exploitation in a production environment.

Overall, while the API provides functional capabilities, remediation of the identified vulnerabilities is critical before deploying to a public-facing environment. Continuous security testing and adherence to secure API development practices are essential to maintain a robust defense against evolving threats.

End of report